

WEEK 2

TOPICS TO BE COVERED

- Wrapper classes (Integer, Double, Character, etc.)
- Autoboxing and Unboxing
- Enumeration (enum)
- Enum methods and constructors

WRAPPER CLASS

- A Wrapper class in Java is one whose object wraps or contains primitive data types.
- When we create an object in a wrapper class, it contains a field and, in this field, we can store primitive data types.
- In other words, we can wrap a primitive value into a wrapper class object.

WRAPPER CLASS

Primitive type	Wrapper Class
boolean	Boolean
byte	Byte
char	Character
float	Float
int	Integer
long	Long
short	Short
double	Double

Need for WRAPPER CLASS

- They convert primitive data types into objects. Objects are needed if we wish to modify the arguments passed into a method (because primitive types are passed by value).
- The classes in `java.util` package handle only objects and hence wrapper classes help in this case.
- Data structures in the Collection framework, such as `ArrayList` and `Vector`, store only objects (reference types) and not primitive types.
- An object is needed to support synchronization in multithreading.

```
class wrapperDemo
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        // Primitive data type
```

```
        int b;
```

```
        // Integer wrapper class
```

```
        Integer a;
```

```
        // assigning value to primitive
```

```
        b = 357;
```

```
        // auto-boxing or auto wrapping converting primitive int to Integer object
```

```
        a = b;
```

```
        System.out.println("The primitive int b is: " + b);
```

```
        System.out.println("The Integer object a is: " + a);
```

```
    }
```

```
}
```

Example

OUTPUT:

The primitive int b is: 357

The Integer object a is: 357

AUTOBOXING AND UNBOXING

- **Autoboxing**
- The automatic conversion of primitive types to the object of their corresponding wrapper classes is known as autoboxing.
- For example: **conversion of int to Integer**, long to Long, double to Double, etc.

AUTOBOXING AND UNBOXING

- **Unboxing**
- It is just the reverse process of autoboxing. Automatically converting an object of a wrapper class to its corresponding primitive type is known as unboxing.
- For example, **conversion of Integer to int**, Long to long, Double to double, etc.

```
public class AutoBoxingUnboxingDemo {  
    public static void main(String[] args) {  
  
        // Autoboxing: primitive to wrapper object  
        int num = 50;  
        Integer intObj = num; // Autoboxing  
  
        System.out.println("Autoboxing Example:");  
        System.out.println("Primitive value : " + num);  
        System.out.println("Wrapper object : " + intObj);  
  
        // Unboxing: wrapper object to primitive  
        Integer obj = 100;  
        int value = obj;    // Unboxing  
  
        System.out.println("\nUnboxing Example:");  
        System.out.println("Wrapper object : " + obj);  
        System.out.println("Primitive value : " + value);  
    }  
}
```

Example

OUTPUT

Autoboxing Example:
Primitive value : 50
Wrapper object : 50

Unboxing Example:
Wrapper object : 100
Primitive value : 100

ENUMERATION

- In Java, enumerations (enums) are a special type used to define a group of named constants.
- Enums help in readability, maintainability, and type safety in programs by assigning meaningful names to integer values.
- Mainly useful when we have a small set of possible values for an item like directions, days of week, etc.
- Syntax

```
enum EnumName{  
    Constant 1, Constant 2, ..., Constant n  
}
```

Example

```
enum TrafficLight {  
    RED, GREEN, YELLOW  
}
```

```
public class Test{  
  
    public static void main(String[] args){  
  
        TrafficLight x = TrafficLight.RED;  
        System.out.println(x);  
    }  
}
```

OUTPUT :

RED

Enum with switch Statement

```
//import java.io.*;

class enumDemo{

    // Enum Declared
    enum Color {
        RED,
        GREEN,
        BLUE,
        YELLOW;
    }
}
```

```
public static void main(String[] args){

    Color var_1 = Color.YELLOW;

    // Switch case with Enum
    switch (var_1) {
        case RED:
            System.out.println("Red color");
            break;
        case GREEN:
            System.out.println("Green color");
            break;
        case BLUE:
            System.out.println("Blue color");
            break;
        default:
            System.out.println("Other color");
    }
}
}
```

Enum methods and constructors

- Enums can have constructors and methods, executed separately for each constant
- An enum can also have a constructor just like a class.
- The constructor is called automatically when the constants are created. You cannot call it yourself.
- Here, each constant in the enum has a value (a string) that is set through the constructor

Enumerations methods

Enum methods

method	description
<code>int compareTo(E)</code>	all enum types are Comparable by order of declaration
<code>boolean equals(o)</code>	not needed; can just use <code>==</code>
<code>String name()</code>	equivalent to <code>toString</code>
<code>int ordinal()</code>	returns an enum's 0-based number by order of declaration (first is 0, then 1, then 2, ...)

method	Description
<code>static E valueOf(s)</code>	converts a string into an enum value
<code>static E[] values()</code>	an array of all values of your enumeration

```
enum Color{
    RED, GREEN, BLUE;

    private Color(){

        System.out.println("Constructor called for: " + this);
    }

    public void display(){

        System.out.println("Color is: " + this);
    }
}

public class Test{

    public static void main(String[] args){

        Color c1 = Color.RED;
        c1.display();
    }
}
```

Example 1:

OUTPUT

```
Constructor called for: RED
Constructor called for: GREEN
Constructor called for: BLUE
Color is: RED
```

